



# Universidad Mariano Gálvez De Guatemala

---

Universidad Mariano Gálvez De Guatemala Centro universitario de San Juan Sacatepéquez

**Facultad:** De Ingeniería En Sistemas De Información y Ciencias De la Computación

**Docente:** Ing. Miguel Ángel Catalán

**Curso:** Algoritmos

**Ciclo:** II

*Proyecto*

## **Manual Técnico**

*Gestor De Notas Académicas*

**Nombre:** Derek Stuard Jimenez Deleón

**Carné:** 7590-25-10502

**Sección:** B

**Fecha:** 17/10/2025

## **TABLA DE CONTENIDO**

|           |                                                                     |           |
|-----------|---------------------------------------------------------------------|-----------|
| <b>1.</b> | <b>Descripción técnica general del sistema .....</b>                | <b>3</b>  |
| <b>2.</b> | <b>Estructura general del código.....</b>                           | <b>3</b>  |
| 2.1.      | Función principal del menú (mostrar_menu).....                      | 3         |
| 2.2.      | Ciclo principal de ejecución (main) .....                           | 3         |
| 2.3.      | Estructuras de datos utilizadas .....                               | 4         |
| 2.4.      | Registro de cursos (registrar_curso).....                           | 4         |
| 2.5.      | Visualización de cursos (mostrar_cursos) .....                      | 4         |
| 2.6.      | Cálculo de promedio general (calcular_promedio) .....               | 4         |
| 2.7.      | Conteo de aprobados y reprobados (contar_aprobados_reprobados)..... | 4         |
| 2.8.      | Búsqueda de curso por nombre (buscar_curso_lineal) .....            | 4         |
| 2.9.      | Actualización de nota (actualizar_nota).....                        | 5         |
| 2.10      | . Eliminación de curso (eliminar_curso) .....                       | 5         |
| 2.11.     | Ordenamiento de cursos .....                                        | 5         |
| 2.12.     | Búsqueda binaria por código (buscar_curso_binaria) .....            | 5         |
| 2.13.     | Simulación de cola de revisiones (simular_cola_revisiones) .....    | 5         |
| 2.14.     | Historial de cambios (mostrar_historial_cambios) .....              | 5         |
| 2.15.     | Ejecución del programa.....                                         | 6         |
| <b>3.</b> | <b>Uso de estructuras de datos .....</b>                            | <b>6</b>  |
| 3.1.      | Lista principal de cursos (cursos).....                             | 6         |
| 3.2.      | Pila de historial de cambios (historial_cambios) .....              | 7         |
| 3.3.      | Cola de solicitudes de revisión (cola_revisiones) .....             | 7         |
| <b>4.</b> | <b>Justificación de los Algoritmos de Ordenamiento .....</b>        | <b>8</b>  |
| 4.1.      | Ordenamiento Burbuja (por nota) .....                               | 8         |
| 4.2.      | Ordenamiento por Inserción (por nombre) .....                       | 9         |
| <b>5.</b> | <b>Documentación de las funciones Principales .....</b>             | <b>10</b> |
| 5.1.      | mostrar_menu() .....                                                | 10        |
| 5.2.      | main().....                                                         | 10        |
| 5.3.      | registrar_curso() .....                                             | 10        |
| 5.4.      | mostrar_cursos().....                                               | 10        |
| 5.5.      | calcular_promedio().....                                            | 11        |
| 5.6.      | contar_aprobados_reprobados().....                                  | 11        |
| 5.7.      | buscar_curso_lineal() .....                                         | 11        |

|                                                               |    |
|---------------------------------------------------------------|----|
| 5.8. actualizar_nota()                                        | 11 |
| 5.9. eliminar_curso()                                         | 11 |
| 5.10. ordenar_por_nota()                                      | 11 |
| 5.11. ordenar_por_nombre()                                    | 12 |
| 5.12. buscar_curso_binaria()                                  | 12 |
| 5.13. simularColaRevisiones()                                 | 12 |
| 5.14. mostrar_historial_cambios()                             | 12 |
| 5.15. Bloque de ejecución (if __name__ == "__main__": main()) | 12 |
| 6. Diagrama general o pseudocódigo principal                  | 13 |

## **1. Descripción técnica general del sistema**

El sistema es un gestor de notas académicas que permite a los usuarios registrar cursos y sus notas, mostrar los cursos registrados, calcular el promedio general de las notas, contar cursos aprobados y reprobados, buscar cursos por nombre o código, actualizar notas, eliminar cursos, ordenar cursos por nota o nombre, simular una cola de solicitudes de revisión de notas y mantener un historial de cambios utilizando una pila. El sistema ofrece una interfaz de menú para que los usuarios interactúen con las diferentes funcionalidades. Lo que permite que el usuario tenga una experiencia amigable e intuitiva para gestionar sus notas académicas de manera eficiente. El sistema está implementado en Python y utiliza estructuras de datos como listas para almacenar los cursos, pila para el historial de cambios y una lista para simular la cola de solicitudes de revisión de notas. Este proyecto fue desarrollado como parte del curso de Algoritmos para que el estudiante pueda aplicar los conceptos aprendidos en clase. Eso permite al estudiante reforzar sus habilidades de programación y comprensión de estructuras de datos y algoritmos fundamentales en Python.

## **2. Estructura general del código**

El código se organiza en funciones independientes que se invocan desde un menú principal. Cada opción del menú realiza una operación específica, como registrar un curso, calcular el promedio o buscar un curso. Se utiliza una lista principal llamada 'cursos' para almacenar la información, una pila llamada 'historial\_cambios' para registrar las modificaciones realizadas, y una cola llamada 'cola\_revisiones' para simular solicitudes de revisión.

### **2.1. Función principal del menú (mostrar\_menu)**

El programa inicia mostrando un menú principal al usuario mediante la función `mostrar_menu()`.

Esta función presenta en pantalla todas las opciones numéricas que el usuario puede seleccionar, desde registrar cursos hasta salir del programa.

El menú es el punto de acceso a todas las funcionalidades y mantiene una interfaz de texto simple pero estructurada.

### **2.2. Ciclo principal de ejecución (main)**

La función `main()` es el núcleo del programa.

Dentro de un bucle `while True`, se llama a `mostrar_menu()` y se captura la opción elegida por el usuario.

Dependiendo de la entrada, el programa ejecuta una de las funciones específicas.

Este ciclo permite mantener la interacción continua con el usuario hasta que seleccione la opción “Salir”.

El control de flujo se basa en una estructura condicional (`if-elif`) que dirige la ejecución hacia la operación correspondiente.

### 2.3. Estructuras de datos utilizadas

El sistema se apoya en tres estructuras principales:

- **cursos**: lista principal donde cada elemento es un diccionario con las claves código, nombre y nota.  
Sirve como base de datos interna para almacenar toda la información académica.
- **historial\_cambios**: lista que actúa como **pila (LIFO)** para guardar los registros de acciones realizadas (altas, modificaciones o eliminaciones).
- **cola\_revisiones**: lista que funciona como **cola (FIFO)**, utilizada para gestionar las solicitudes de revisión de notas.

Estas estructuras son compartidas y manipuladas por todas las funciones del sistema.

### 2.4. Registro de cursos (registrar\_curso)

Esta función permite crear nuevos registros dentro de la lista cursos.

Solicita al usuario ingresar un código, nombre y nota, verificando que el código no esté duplicado y que la nota sea válida (0–100).

Una vez validados los datos, el curso se almacena en la lista y se añade una entrada al historial de cambios.

### 2.5. Visualización de cursos (mostrar\_cursos)

Permite mostrar todos los cursos registrados con su código, nombre y nota.

Recorre la lista cursos y presenta los datos en formato tabular o en texto consecutivo.

Si no existen cursos registrados, muestra un mensaje indicativo.

### 2.6. Cálculo de promedio general (calcular\_promedio)

Obtiene el promedio aritmético de todas las notas registradas en la lista cursos.

Suma las notas y las divide entre la cantidad total de cursos.

Si no hay cursos, evita el cálculo e informa al usuario.

### 2.7. Conteo de aprobados y reprobados (contar\_aprobados\_reprobados)

Evalúa cada curso en la lista y determina si está aprobado ( $\text{nota} \geq 60$ ) o reprobado ( $\text{nota} < 60$ ).

Presenta en pantalla el total de cursos en cada categoría.

### 2.8. Búsqueda de curso por nombre (buscar\_curso\_lineal)

Realiza una búsqueda lineal dentro de la lista cursos, comparando el nombre ingresado por el usuario con los nombres almacenados.

Si encuentra coincidencia, muestra la información del curso correspondiente.

### 2.9. Actualización de nota (**actualizar\_nota**)

Busca un curso por su código y permite modificar su nota actual.

Después de actualizar el valor, registra el cambio en la pila `historial_cambios` para mantener trazabilidad de las modificaciones realizadas.

### 2.10. Eliminación de curso (**eliminar\_curso**)

Elimina un curso existente identificado por su código.

El registro eliminado se guarda como texto en `historial_cambios` para mantener un registro del evento.

Si no se encuentra el código, informa al usuario.

### 2.11. Ordenamiento de cursos

El programa incluye dos métodos de ordenamiento independientes:

- **Por nota (`ordenar_por_nota`):** utiliza el método de **burbuja**, ordenando los cursos de menor a mayor según su nota.
- **Por nombre (`ordenar_por_nombre`):** aplica el método de **inserción**, organizando los cursos alfabéticamente.

Ambas funciones reorganizan la lista principal y muestran los resultados actualizados.

### 2.12. Búsqueda binaria por código (**buscar\_curso\_binaria**)

Permite localizar un curso utilizando el algoritmo de **búsqueda binaria**, lo que mejora la eficiencia en comparación con la búsqueda lineal.

Antes de la búsqueda, la lista se ordena según los códigos para garantizar la correcta ejecución del algoritmo.

### 2.13. Simulación de cola de revisiones (**simular\_cola\_revisiones**)

Emula el funcionamiento de una **cola FIFO** para manejar solicitudes de revisión de notas.

Ofrece un submenú interno con las siguientes opciones:

1. Agregar solicitud
2. Procesar solicitud (eliminar la primera en la cola)
3. Mostrar solicitudes en espera
4. Salir del submenú

Cada solicitud se almacena y procesa en el orden en que fue ingresada.

### 2.14. Historial de cambios (**mostrar\_historial\_cambios**)

Permite visualizar las acciones registradas en la pila `historial_cambios`.

Los eventos se muestran en orden inverso (del más reciente al más antiguo), simulando el comportamiento real de una pila.

## 2.15. Ejecución del programa

El bloque final del código:

```
if __name__ == "__main__":  
    main()
```

indica el punto de inicio del programa.

Cuando el archivo se ejecuta directamente, llama automáticamente a la función `main()` e inicia el ciclo del menú principal.

## 3. Uso de estructuras de datos

- **Listas:** almacenan los cursos con sus atributos (código, nombre, nota).
- **Pilas:** registran los cambios realizados, permitiendo consultar el historial en orden inverso.
- **Colas:** simulan solicitudes de revisión, procesando los elementos en orden de llegada (FIFO).

### 3.1. Lista principal de cursos (cursos)

La lista cursos es la estructura fundamental del programa.

En ella se guarda toda la información relacionada con las asignaturas registradas por el usuario.

Cada elemento de la lista es un **diccionario** que almacena tres datos esenciales:

- código: identificador único del curso.
- nombre: nombre o descripción del curso.
- nota: calificación obtenida por el estudiante.

#### Funciones que utilizan esta lista:

- **registrar\_curso():** agrega un nuevo diccionario a la lista, validando que el código no esté duplicado.
- **mostrar\_cursos():** recorre y muestra todos los elementos de la lista.
- **calcular\_promedio():** recorre la lista sumando las notas para obtener el promedio general.
- **contar\_aprobados\_reprobados():** evalúa cada nota para clasificar los cursos en aprobados o reprobados.
- **buscar\_curso\_lineal():** realiza una búsqueda secuencial comparando nombres de cursos.
- **actualizar\_nota() y eliminar\_curso():** localizan un curso mediante su código para modificar o eliminar su registro.

- **ordenar\_por\_nota()** y **ordenar\_por\_nombre()**: aplican los algoritmos de **burbuja** e **inserción**, respectivamente, para organizar los cursos.
- **buscar\_curso\_binaria()**: ordena la lista por código y utiliza una búsqueda binaria para localizar un curso de manera más eficiente.

**Importancia:** Esta lista actúa como la **base de datos temporal del sistema**, ya que contiene toda la información académica manipulada durante la ejecución del programa.

### 3.2. Pila de historial de cambios (historial\_cambios)

El programa implementa una **pila (LIFO – Last In, First Out)** utilizando una lista común de Python.

Esta estructura almacena los registros de cada acción importante que realiza el usuario dentro del sistema, como agregar, actualizar o eliminar cursos.

Cada vez que ocurre un cambio, se **agrega un texto descriptivo** al final de la lista, simulando el comportamiento de una pila.

Cuando se consulta el historial, los registros se muestran en **orden inverso**, reflejando el principio “último en entrar, primero en salir”.

**Funciones que utilizan esta pila:**

- **registrar\_curso()**: añade un mensaje cuando se crea un nuevo curso.
- **actualizar\_nota()**: registra el cambio de nota con la información anterior y la nueva.
- **eliminar\_curso()**: guarda un mensaje con los datos del curso eliminado.
- **mostrar\_historial\_cambios()**: recorre la pila en orden inverso y muestra las acciones más recientes primero.

**Importancia:** La pila permite **rastrear las modificaciones** realizadas en el sistema, funcionando como un **registro histórico o bitácora** de las acciones del usuario. Este enfoque favorece la transparencia y el control de cambios dentro del programa.

### 3.3. Cola de solicitudes de revisión (cola\_revisiones)

El programa también utiliza una **cola (FIFO – First In, First Out)** implementada con una lista.

Esta estructura se emplea para **simular un sistema de atención de solicitudes de revisión de notas**, donde los cursos se procesan en el mismo orden en que fueron solicitados.

Las operaciones principales que se realizan sobre esta cola son:

- **Encolado (agregar)**: cuando el usuario registra una nueva solicitud de revisión.



- **Desencolado (procesar):** cuando el sistema atiende la primera solicitud en la cola.
- **Visualización:** permite observar todas las solicitudes que aún están pendientes de revisión.

#### Función que gestiona la cola:

- **simularColaRevisiones():** contiene un submenú con opciones para agregar, procesar y mostrar las solicitudes. Cada solicitud corresponde al código de un curso, y el orden de procesamiento sigue estrictamente la lógica FIFO.

**Importancia:** La cola modela un proceso **secuencial y justo**, donde las solicitudes más antiguas se procesan antes que las nuevas, reproduciendo el comportamiento de un sistema de atención en tiempo real.

#### Resumen del uso de estructuras de datos

| Estructura            | Nombre en el código | Tipo de acceso                      | Propósito principal               | Funciones relacionadas                                                                       |
|-----------------------|---------------------|-------------------------------------|-----------------------------------|----------------------------------------------------------------------------------------------|
| Lista de diccionarios | cursos              | Acceso secuencial y directo         | Almacenar cursos, nombres y notas | Todas las funciones principales                                                              |
| Pila (LIFO)           | historial_cambios   | Último en entrar, primero en salir  | Registrar las acciones realizadas | registrar_curso(),<br>actualizar_nota(),<br>eliminar_curso(),<br>mostrar_historial_cambios() |
| Cola (FIFO)           | cola_revisiones     | Primero en entrar, primero en salir | Gestionar solicitudes de revisión | simularColaRevisiones()                                                                      |

## 4. Justificación de los Algoritmos de Ordenamiento

En el sistema **Gestor de Notas Académicas**, se implementan dos algoritmos clásicos de ordenamiento: **el método de burbuja y el método de inserción**.

Ambos fueron seleccionados por su **simplicidad, facilidad de comprensión y aplicabilidad** en listas pequeñas o medianas, que es el tamaño de datos esperado dentro de este programa.

A continuación, se detalla el uso y la justificación de cada uno.

### 4.1. Ordenamiento Burbuja (por nota)

**Función utilizada:** ordenar\_por\_nota()

El algoritmo de **burbuja** se aplica para ordenar los cursos en función de la nota obtenida por el estudiante, de menor a mayor.

El método compara pares consecutivos de elementos dentro de la lista y los intercambia si están en el orden incorrecto. Este proceso se repite hasta que toda la lista queda ordenada.

#### **Justificación del uso:**

1. **Facilidad de implementación:**

Es uno de los algoritmos más sencillos de comprender, lo que lo hace ideal para proyectos académicos o didácticos.

Su estructura basada en bucles anidados permite visualizar claramente el proceso de intercambio entre elementos.

2. **Adecuado para listas pequeñas:**

Dado que el gestor no maneja grandes volúmenes de datos (solo algunos cursos por estudiante), el tiempo de ejecución del algoritmo no afecta el rendimiento general del programa.

3. **Claridad en el comportamiento:**

Permite observar de manera simple cómo las notas “suben” o “bajan” hasta llegar a su posición correcta, lo que ayuda a entender el proceso de ordenamiento paso a paso.

4. **Estabilidad del ordenamiento:**

El método de burbuja conserva el orden relativo entre elementos con la misma nota, lo que evita que se alteren otros datos asociados al curso.

## **4.2. Ordenamiento por Inserción (por nombre)**

**Función utilizada:** `ordenar_por_nombre()`

El algoritmo de **inserción** se utiliza para ordenar los cursos alfabéticamente por nombre. Este método toma cada elemento y lo inserta en su posición correcta dentro de la parte ya ordenada de la lista, desplazando los elementos que sean necesarios.

#### **Justificación del uso:**

1. **Eficiencia en listas parcialmente ordenadas:**

El algoritmo de inserción funciona de manera muy eficiente cuando la lista está casi ordenada, ya que realiza pocos movimientos.

Esto es común en este sistema, donde los cursos se registran en orden y solo requieren ajustes menores.

2. **Control manual del proceso:**

Su estructura permite comprender con claridad cómo se comparan y reubican los elementos, lo que resulta ideal para propósitos educativos.

3. **Menor número de intercambios innecesarios:**

A diferencia del método burbuja, el algoritmo de inserción realiza menos movimientos, lo que optimiza ligeramente el rendimiento en listas cortas.

4. **Ordenamiento estable y seguro:**

Mantiene la coherencia de los datos, asegurando que los cursos con el mismo nombre no se vean afectados por el proceso de reubicación.

## 5. Documentación de las funciones Principales

### 5.1. mostrar\_menu()

**Propósito:** Despliega en pantalla el menú principal del programa con las trece opciones disponibles, que incluyen operaciones como registrar cursos, calcular promedios, ordenar datos o salir del sistema.

**Importancia:** Permite la interacción directa con el usuario y sirve como punto de acceso a todas las demás funciones.

---

### 5.2. main()

**Propósito:** Controla el flujo principal de ejecución del programa. Dentro de un ciclo continuo, lee la opción seleccionada por el usuario e invoca la función correspondiente.

**Importancia:** Es el centro lógico del sistema, ya que coordina todas las operaciones del gestor y mantiene la ejecución activa hasta que el usuario decida salir.

---

### 5.3. registrar\_curso()

**Propósito:** Permite ingresar nuevos cursos con su respectivo código, nombre y nota. Valida que el código no exista previamente y que la nota esté dentro del rango permitido (0–100).

**Importancia:** Constituye la base del sistema, pues crea los registros que alimentan la lista principal de cursos.

---

### 5.4. mostrar\_cursos()

**Propósito:** Muestra todos los cursos almacenados en la lista principal, junto con su código, nombre y nota.

**Importancia:** Facilita la visualización del contenido actual del sistema y permite verificar los datos registrados.

---

### 5.5. calcular\_promedio()

**Propósito:** Calcula el promedio general de todas las notas ingresadas en el sistema.

**Importancia:** Permite obtener un indicador general del rendimiento académico de los cursos registrados.

---

### 5.6. contar\_aprobados\_reprobados()

**Propósito:** Cuenta la cantidad de cursos aprobados (nota mayor o igual a 60) y reprobados (nota menor a 60).

**Importancia:** Proporciona un resumen cuantitativo del desempeño académico global.

---

### 5.7. buscar\_curso\_lineal()

**Propósito:**

Busca un curso por su nombre dentro de la lista principal mediante un recorrido secuencial.

**Importancia:**

Permite localizar rápidamente un curso sin necesidad de conocer su código, aplicando el método de búsqueda lineal.

---

### 5.8. actualizar\_nota()

**Propósito:** Modifica la nota de un curso existente identificado por su código.

Registra la acción en el historial de cambios.

**Importancia:** Garantiza la actualización de los datos y mantiene un registro de las modificaciones realizadas.

---

### 5.9. eliminar\_curso()

**Propósito:** Elimina un curso del registro utilizando su código como identificador.

El evento se almacena en la pila de historial de cambios.

**Importancia:** Permite mantener actualizada la base de datos, eliminando registros innecesarios o erróneos.

---

### 5.10. ordenar\_por\_nota()

**Propósito:** Ordena los cursos de menor a mayor nota utilizando el algoritmo de burbuja.

**Importancia:** Facilita la comparación y análisis del rendimiento académico al organizar los datos según las calificaciones.

---

#### **5.11. ordenar\_por\_nombre()**

**Propósito:** Ordena los cursos alfabéticamente por nombre aplicando el algoritmo de inserción.

**Importancia:** Permite visualizar los cursos de forma estructurada y coherente, favoreciendo la búsqueda y presentación ordenada.

---

#### **5.12. buscar\_curso\_binaria()**

**Propósito:** Localiza un curso mediante su código aplicando el método de búsqueda binaria, después de ordenar la lista.

**Importancia:** Demuestra una búsqueda eficiente que reduce el número de comparaciones necesarias para encontrar un curso específico.

---

#### **5.13. simular\_cola\_revisiones()**

**Propósito:** Simula una cola FIFO para manejar solicitudes de revisión de notas. Permite agregar, procesar o visualizar las solicitudes pendientes.

**Importancia:** Reproduce un proceso real de atención por orden de llegada, aplicando el concepto de colas en programación.

---

#### **5.14. mostrar\_historial\_cambios()**

**Propósito:** Muestra los registros almacenados en la pila de historial de cambios, presentando las acciones más recientes primero.

**Importancia:** Proporciona una trazabilidad clara de todas las modificaciones realizadas, fortaleciendo el control interno del sistema.

---

#### **5.15. Bloque de ejecución (if \_\_name\_\_ == "\_\_main\_\_": main())**

**Propósito:** Indica el punto de inicio del programa. Al ejecutar el archivo directamente, se activa la función main() y se inicia el menú principal.

**Importancia:** Garantiza que el sistema funcione de forma autónoma y controlada desde su propia ejecución.

## 6. Diagrama general o pseudocódigo principal

INICIO PROGRAMA

```
// Declaración de estructuras de datos globales
DECLARAR cursos = LISTA_VACÍA (Repositorio principal)
DECLARAR historial_cambios = LISTA_VACÍA (Pila)
DECLARAR cola_revisiones = LISTA_VACÍA (Cola)
```

FUNCION main():

MIENTRAS VERDADERO:

LLAMAR mostrar\_menu()  
LEER opcion\_seleccionada

// Evaluar la opción seleccionada

SELECCIONAR CASO opcion\_seleccionada:

CASO "1":

LLAMAR registrar\_curso()

CASO "2":

LLAMAR mostrar\_cursos()

CASO "3":

LLAMAR calcular\_promedio()

CASO "4":

LLAMAR contar\_aprobados\_reprobados()

CASO "5":

LLAMAR buscar\_curso\_lineal()

CASO "6":

LLAMAR actualizar\_nota()

CASO "7":

LLAMAR eliminar\_curso()

CASO "8":

LLAMAR ordenar\_por\_nota() (Burbuja)

CASO "9":

LLAMAR ordenar\_por\_nombre() (Inserción)

CASO "10":

LLAMAR buscar\_curso\_binaria()

CASO "11":

LLAMAR simular\_cola\_revisiones()

CASO "12":

LLAMAR mostrar\_historial\_cambios()

// Opción 13: Salir del programa

CASO "13":

IMPRIMIR "Saliendo del gestor de notas."

SALIR DEL BUCLE

IMPRIMIR "Opción no válida."

FIN MIENTRAS

FIN PROGRAMA